CLD-105 · Orchestration de conteneurs avancé (Kubernetes/Git)

Compte rendu – TP3

Déployer example-voting-app

Élément	Valeur
Étudiant	Qassim Messili
Formation	Mastère (Bac+5) – IPI Lyon
Date	27 octobre 2026
Cluster	Cluster kubeadm du TP2 : `k8s-cp` (10.2.100.225) et `k8s-w1` (10.2.100.219)
Namespace	`vote`
Accès	http://vote.local:30080 et http://result.local:30080

Sommaire

Si le sommaire est vide : clic droit → « Mettre à jour les champs » (ou F9 dans Word).

Introduction

Au TP2, j'ai construit un cluster Kubernetes à deux nœuds. Dans ce TP3, je m'en sers enfin pour faire tourner une vraie application : **example-voting-app**, l'application « fil rouge » du module. C'est une petite application de vote : on vote sur une page, et on voit le résultat sur une autre.

Elle est intéressante parce qu'elle ressemble à une vraie application d'entreprise : cinq morceaux séparés, écrits dans trois langages différents, qui doivent se parler entre eux. Mon travail a été de décrire chaque morceau dans des fichiers YAML, de les déployer, puis de rendre l'application accessible depuis mon PC avec de vrais noms (vote.local et result.local).

J'ai aussi testé ce qui se passe quand on met à jour l'application, quand on la casse exprès, et quand on revient en arrière. Enfin, j'ai constaté un défaut volontaire du TP : les votes sont perdus si la base de données redémarre. Ce problème sera corrigé au TP4.

Les cinq composants de l'application

Composant	Image	Port	Rôle (en simple)
vote	dockersamples/ examplevotingapp_vote	80	La page où l'on vote (Python)
redis	redis:alpine	6379	Une « boîte aux lettres » qui stocke les votes en attente
worker	dockersamples/ examplevotingapp_worker	—	Le facteur : il prend les votes dans redis et les range dans la base (.NET)
db	postgres:15-alpine	5432	La base de données qui garde les votes
result	dockersamples/ examplevotingapp_result	80	La page qui affiche les résultats (Node.js)

Le chemin d'un vote est donc : **vote → redis → worker → db → result**. Si un seul maillon casse, la chaîne s'arrête.

Adaptations à mon environnement

Énoncé	Chez moi	Pourquoi
IP du worker 192.168.56.11	10.2.100.219	Réseau de l'ESXi de l'école
Fichiers fournis dans code/	Fichiers écrits par moi	L'archive TP3.zip était abîmée ; l'énoncé demande de toute façon d'écrire les fichiers
Bonus MetalLB	Non réalisé	Pas de plage d'IP libre et hors DHCP sur le réseau de l'école (voir Partie 7)

> **Convention de nommage :** tous mes objets portent les étiquettes (labels) `app.kubernetes.io/name: <composant>` et `app.kubernetes.io/part-of: voting-app`. Les Services s'appellent obligatoirement `redis` et `db`, car ces noms sont écrits en dur dans le code de l'application.

Partie 0 – Vérifier le cluster

Avant de commencer, j'ai vérifié que mon cluster du TP2 était toujours en bonne santé, puis j'ai créé un dossier `~/tp3` pour ranger tous mes fichiers. Toutes les commandes de ce TP sont lancées depuis le control plane (`k8s-cp`).

VM 1 – k8s-cp

```
kubectl get nodes
kubectl get pods -A | grep -v Running
mkdir -p ~/tp3 && cd ~/tp3
```

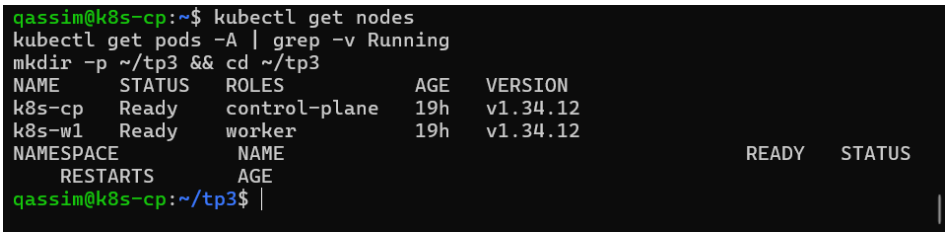


Figure 1 – Les deux nœuds sont Ready et aucun pod n'est en erreur

Les deux nœuds sont **Ready** depuis 19 heures, et la commande `grep -v Running` n'affiche que la ligne de titre : aucun pod n'est en panne. Je peux commencer.

Partie 1 – Le namespace et les composants qui gardent des données

Je commence par les deux composants « avec état » (ils gardent des données) : redis et la base db. Tout est rangé dans un **namespace** `vote`, une sorte de dossier qui isole l'application du reste du cluster.

Les fichiers

00-namespace.yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: vote
  labels:
    app.kubernetes.io/part-of: voting-app
```

Pour redis, le fichier contient deux objets : un **Deployment** (qui fait tourner le conteneur et le relance s'il tombe) et un **Service** (qui donne un nom et une adresse stables pour le joindre).

01-redis.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: vote
  labels:
    app.kubernetes.io/name: redis
    app.kubernetes.io/part-of: voting-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: redis
  template:
    metadata:
      labels:
        app.kubernetes.io/name: redis
        app.kubernetes.io/part-of: voting-app
    spec:
      containers:
      - name: redis
        image: redis:alpine
        ports:
        - name: redis
          containerPort: 6379
---
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: vote
spec:
  type: ClusterIP
  selector:
```

```
    app.kubernetes.io/name: redis
  ports:
  - name: redis
    port: 6379
    targetPort: redis
```

Le fichier de la base db suit le même modèle, avec trois différences : la stratégie **Recreate**, les identifiants PostgreSQL passés en variables d'environnement, et un volume `emptyDir` pour les données.

02-db.yaml (extrait du Deployment ; le Service db est sur le même modèle que redis, port 5432)

```
spec:
  replicas: 1
  strategy:
    type: Recreate
  template:
    spec:
      containers:
      - name: db
        image: postgres:15-alpine
        env:
        - name: POSTGRES_USER
          value: postgres
        - name: POSTGRES_PASSWORD
          value: postgres
        ports:
        - name: postgres
          containerPort: 5432
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
      volumes:
      - name: data
        emptyDir: {}
```

Déploiement et vérification

VM 1

```
kubectl apply -f 00-namespace.yaml -f 01-redis.yaml -f 02-db.yaml
sleep 30
kubectl -n vote get deploy,svc,endpointslices
kubectl -n vote get pods -o wide
kubectl -n vote logs deploy/db | tail -5
```

```
sleep 30
kubectl -n vote get deploy,svc,endpointslices
kubectl -n vote get pods -o wide
kubectl -n vote logs deploy/db | tail -5
namespace/vote created
deployment.apps/redis created
service/redis created
deployment.apps/db created
service/db created
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/db      1/1     1            1           30s
deployment.apps/redis   1/1     1            1           30s

NAME            TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
service/db      ClusterIP   10.103.193.148   <none>        5432/TCP   30s
service/redis   ClusterIP   10.98.206.39     <none>        6379/TCP   30s

NAME                                        ADDRESSTYPE   PORTS   ENDPOINTS
       AGE
endpointslice.discovery.k8s.io/db-ds65f     IPv4          5432    10.244.22
8.73   30s
endpointslice.discovery.k8s.io/redis-6nlgd   IPv4          6379    10.244.22
8.74   30s
NAME                     READY   STATUS    RESTARTS   AGE   IP
NODE     NOMINATED NODE   READINESS GATES
db-596fbb45b6-6vb8m      1/1     Running   0          30s   10.244.228.73
k8s-w1   <none>           <none>
redis-54d64cc577-4qxp4   1/1     Running   0          30s   10.244.228.74
k8s-w1   <none>           <none>
2026-10-27 08:10:10.632 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", p
ort 5432
2026-10-27 08:10:10.632 UTC [1] LOG:  listening on IPv6 address "::", port 5
432
2026-10-27 08:10:10.671 UTC [1] LOG:  listening on Unix socket "/var/run/pos
tgresql/.s.PGSQL.5432"
2026-10-27 08:10:10.725 UTC [55] LOG:  database system was shut down at 2026
-10-27 08:10:10 UTC
2026-10-27 08:10:10.744 UTC [1] LOG:  database system is ready to accept con
nections
qassim@k8s-cp:~/tp3$
```

Figure 2 – redis et db en marche, avec chacun un endpoint

Tout est bon : les deux Deployments sont à **1/1**, les deux Services ont bien un **endpoint** (l'adresse du pod derrière : 10.244.228.73 pour db et .74 pour redis) et les logs de la base se terminent par `database system is ready to accept connections`. Les deux pods tournent sur le worker `k8s-w1`, car le control plane refuse les pods applicatifs (taint vu au TP2).

> **Un endpoint, c'est quoi ?** C'est la liste des pods vers lesquels un Service envoie le trafic. Si cette liste est vide, le Service ne mène nulle part. La cause la plus fréquente est une étiquette (label) mal recopiée entre le Service et les pods.

> **Question 1 – Pourquoi la stratégie Recreate pour db ?** PostgreSQL ne supporte pas que deux copies de lui-même écrivent en même temps dans le même dossier de données. Avec la stratégie par défaut (RollingUpdate), Kubernetes démarre le nouveau pod **avant** d'arrêter l'ancien : pendant un moment, deux bases tourneraient en parallèle, avec un vrai risque d'abîmer les données (surtout au TP4, avec un disque persistant). Avec **Recreate**, Kubernetes arrête d'abord l'ancien pod, puis démarre le nouveau. On accepte une petite coupure de quelques secondes pour être sûr de ne rien casser.

Partie 2 – Les composants de l'application

vote : le front de vote

Pour vote, l'énoncé demande d'écrire noir sur blanc la façon dont les mises à jour doivent se faire. J'ai aussi ajouté une annotation `change-cause`, qui sert d'étiquette dans l'historique des versions.

03-vote.yaml (Deployment ; suivi d'un Service ClusterIP vote sur le port 80)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vote
  namespace: vote
  annotations:
    kubernetes.io/change-cause: "version initiale : Cats / Dogs"
spec:
  replicas: 1
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app.kubernetes.io/name: vote
  template:
    metadata:
      labels:
        app.kubernetes.io/name: vote
        app.kubernetes.io/part-of: voting-app
    spec:
      containers:
      - name: vote
        image: dockersamples/examplevotingapp_vote
        env:
        - name: OPTION_A
          value: Cats
        - name: OPTION_B
          value: Dogs
        ports:
        - name: http
          containerPort: 80
```

Réglage	Ce qu'il veut dire
`maxSurge: 1`	Pendant une mise à jour, on peut avoir **un pod de plus** que prévu.
`maxUnavailable: 0`	On n'a **jamais le droit de perdre** un pod qui marche : l'ancien n'est supprimé que quand le nouveau est prêt.
`revisionHistoryLimit: 5`	Kubernetes garde les **5 dernières versions** pour pouvoir revenir en arrière.

result et worker

Le fichier `04-result.yaml` suit la même structure (Deployment et Service sur le port 80), sans variables. Le worker, lui, n'a **pas de Service**, et je l'ai expliqué directement dans le fichier :

05-worker.yaml (début)

```
# Le worker n'a pas de Service : il n'expose aucun port et ne reçoit aucune
# connexion. C'est un simple client qui lit les votes dans redis et les écrit
# dans db. Un Service n'aurait aucun endpoint utile vers lequel router du trafic.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: worker
  ...
```

Pour tester avant de mettre en place l'Ingress, j'ai créé deux Services **NodePort temporaires** (fichier `05b-nodeport-temporaire.yaml`) : ils ouvrent les ports 31000 (vote) et 31001 (result) directement sur les nœuds.

VM 1

```
kubectl apply -f 03-vote.yaml -f 04-result.yaml -f 05-worker.yaml -f 05b-
nodeport-temporaire.yaml
kubectl -n vote rollout status deployment/vote
kubectl -n vote get pods -o wide
kubectl -n vote get svc
```

```
kubectl -n vote rollout status deployment/worker
kubectl -n vote get pods -o wide
kubectl -n vote get svc
deployment.apps/vote created
service/vote created
deployment.apps/result created
service/result created
deployment.apps/worker created
service/vote-np created
service/result-np created
Waiting for deployment "vote" rollout to finish: 0 of 1 updated replicas are
 available...
deployment "vote" successfully rolled out
Waiting for deployment "result" rollout to finish: 0 of 1 updated replicas a
re available...
deployment "result" successfully rolled out
Waiting for deployment "worker" rollout to finish: 0 of 1 updated replicas a
re available...
deployment "worker" successfully rolled out
NAME                      READY   STATUS    RESTARTS   AGE     IP
   NODE     NOMINATED NODE   READINESS GATES
db-596fbb45b6-6vb8m       1/1     Running   0          9m19s   10.244.228.73
   k8s-w1   <none>           <none>
redis-54d64cc577-4qxp4    1/1     Running   0          9m19s   10.244.228.74
   k8s-w1   <none>           <none>
result-556df44dc7-x9cwp   1/1     Running   0          23s     10.244.228.76
   k8s-w1   <none>           <none>
vote-5d8bc7c85b-ghdqx     1/1     Running   0          23s     10.244.228.75
   k8s-w1   <none>           <none>
worker-66d54b4447-9m56k   1/1     Running   0          23s     10.244.228.77
   k8s-w1   <none>           <none>
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
db           ClusterIP   10.103.193.148   <none>        5432/TCP       9m19s
redis        ClusterIP   10.98.206.39     <none>        6379/TCP       9m19s
result       ClusterIP   10.97.158.77     <none>        80/TCP         23s
result-np    NodePort    10.108.119.107   <none>        80:31001/TCP   23s
vote         ClusterIP   10.104.126.191   <none>        80/TCP         23s
vote-np      NodePort    10.101.144.215   <none>        80:31000/TCP   23s
qassim@k8s-cp:~/tp3$
```

Figure 3 – Les cinq pods Running et les Services, dont les deux NodePorts temporaires

Les **cinq pods sont Running**, tous sur `k8s-w1`. On voit les quatre Services ClusterIP et les deux NodePorts temporaires sur 31000 et 31001.

Premier test dans le navigateur

J'ai ouvert les deux pages depuis mon PC, en passant par l'IP du worker :

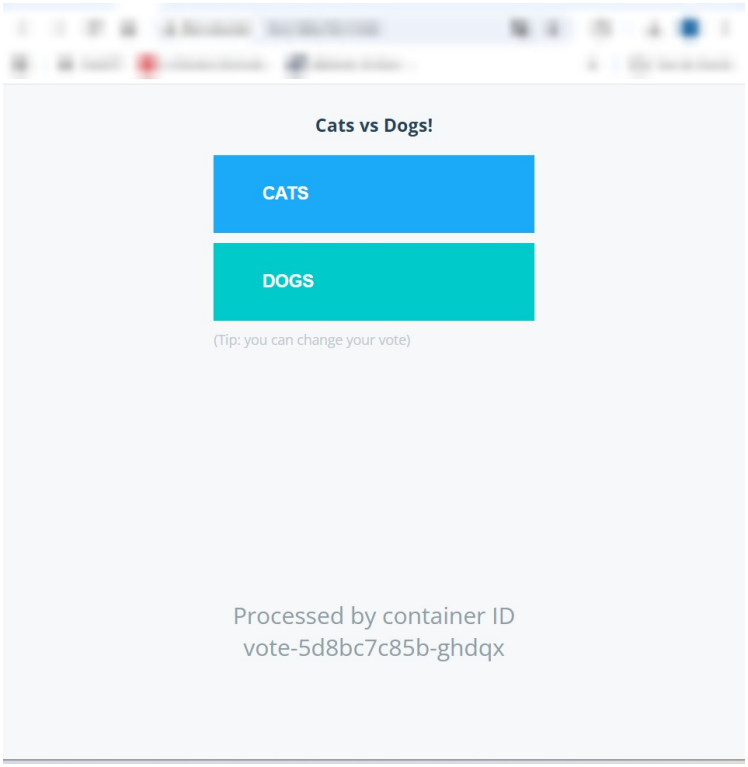


Figure 4 – La page de vote sur http://10.2.100.219:31000

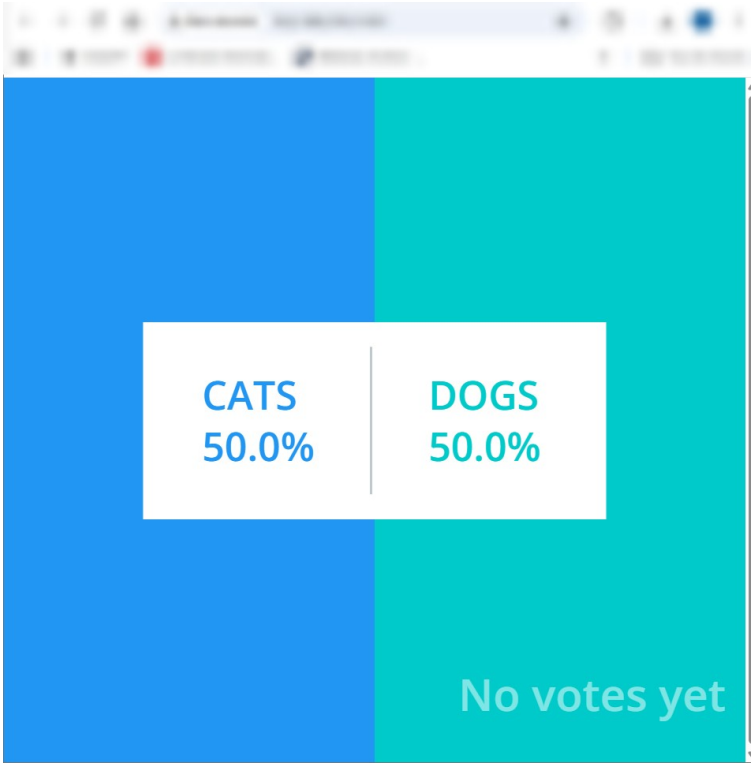


Figure 5 – La page des résultats avant le premier vote

Les options Cats et Dogs viennent bien des variables `OPTION_A` et `OPTION_B`, et la page indique le pod qui a répondu. Côté résultats, « No votes yet » : normal, personne n'a encore voté.

J'ai ensuite voté plusieurs fois. Petite astuce : l'application retient un identifiant par navigateur, il faut donc une **fenêtre de navigation privée** pour chaque nouveau vote.

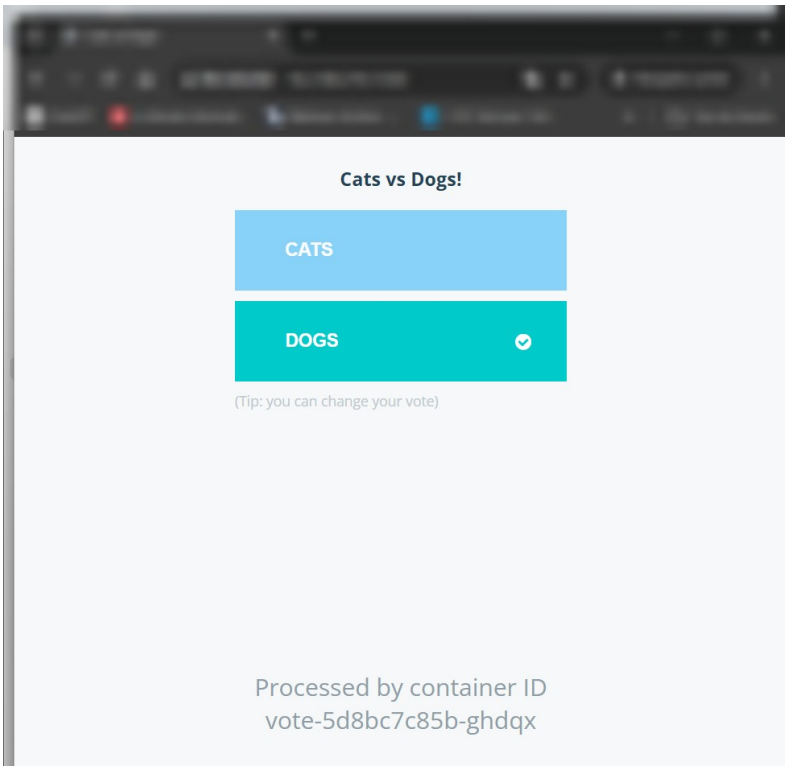


Figure 6 – Un vote Dogs enregistré dans une fenêtre privée

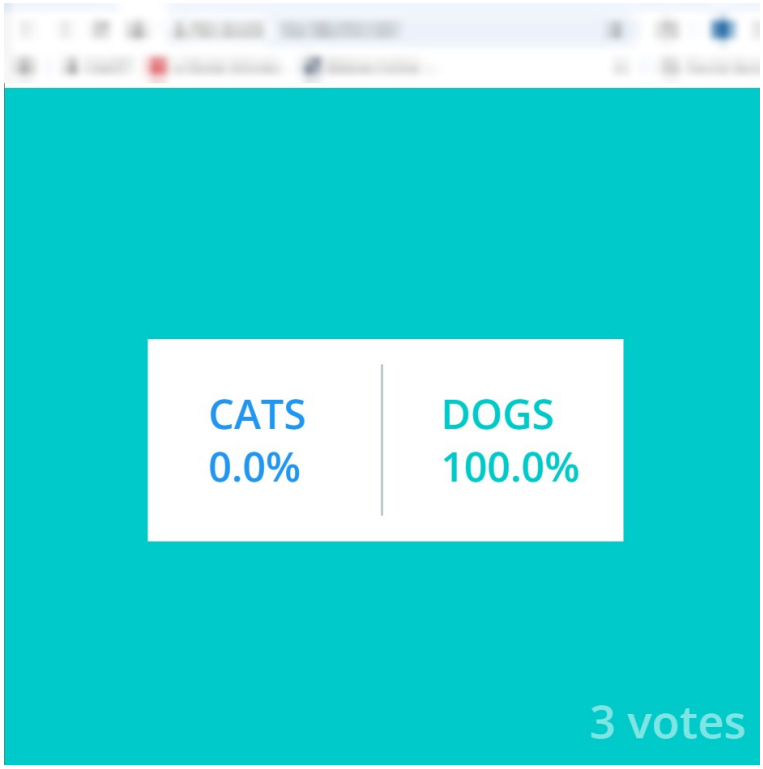


Figure 7 – La page des résultats compte 3 votes

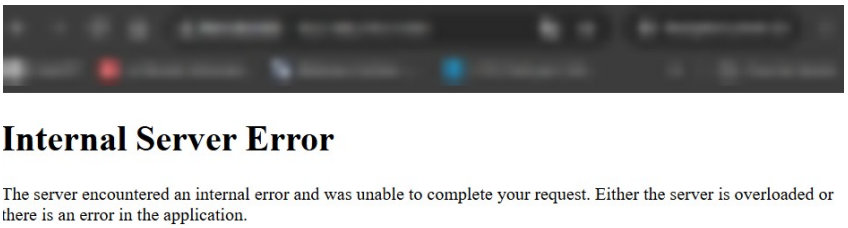
La page des résultats affiche **3 votes**. C'est la preuve que toute la chaîne fonctionne : la page de vote a déposé les votes dans redis, le worker les a pris et rangés dans la base, et la page de résultats les a lus dans la base.

Question 2 : et si on renomme le Service redis ?

Pour cette expérience, j'ai créé un Service identique à redis mais nommé `cache` (fichier `01b-redis-cache.yaml`), puis supprimé le Service `redis`. J'ai préféré un fichier séparé pour pouvoir revenir en arrière facilement.

VM 1

```
kubectl apply -f 01b-redis-cache.yaml
kubectl -n vote delete svc redis
# vote depuis une fenêtre privée, puis :
kubectl -n vote logs deploy/vote --tail=5
```



Internal Server Error

The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

Figure 8 – Le vote provoque une erreur 500

```
qassim@k8s-cp:~/tp3$ kubectl -n vote logs deploy/vote --tail=5
    connection.connect()
  File "/usr/local/lib/python3.11/site-packages/redis/connection.py", line 3
63, in connect
    raise ConnectionError(self._error_message(e))
redis.exceptions.ConnectionError: Error -2 connecting to redis:6379. Name or
 service not known.
10.2.100.219 - - [27/Oct/2026:08:53:06 +0000] "POST / HTTP/1.1" 500 265 "htt
p://10.2.100.219:31000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWeb
Kit/537.36 (KHTML, like Gecko)                  Safari/537.36"
qassim@k8s-cp:~/tp3$ kubectl apply -f 01-redis.yaml
kubectl -n vote delete -f 01b-redis-cache.yaml
kubectl -n vote get svc
deployment.apps/redis unchanged
service/redis created
service "cache" deleted from vote namespace
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
db           ClusterIP   10.103.193.148   <none>        5432/TCP       43m
redis        ClusterIP   10.102.155.132   <none>        6379/TCP       0s
result       ClusterIP   10.97.158.77     <none>        80/TCP         34m
result-np    NodePort    10.108.119.107   <none>        80:31001/TCP   34m
vote         ClusterIP   10.104.126.191   <none>        80/TCP         34m
vote-np      NodePort    10.101.144.215   <none>        80:31000/TCP   34m
qassim@k8s-cp:~/tp3$
```

Figure 9 – Logs de vote : le nom redis est introuvable, puis retour du Service redis

Question 2 – Que se passe-t-il ? La page de vote s'affiche encore (l'affichage n'a pas besoin de redis), mais **chaque vote provoque une erreur 500**. Les logs de vote expliquent pourquoi : `Error -2 connecting to redis:6379. Name or service not known`. Le programme cherche une machine qui s'appelle exactement `redis`. Ce nom n'existe plus, le DNS du cluster ne trouve rien. **Le**

> **nom du Service est donc un contrat avec le code de l'application** : on ne peut pas le changer librement.

J'ai ensuite remis le Service `redis` avec `kubectl apply -f 01-redis.yaml`. Détail intéressant : le Service recréé a reçu une **nouvelle adresse IP** (10.102.155.132 au lieu de 10.98.206.39), et pourtant le vote a remarché tout de suite. L'application ne connaît pas l'IP, elle passe par le **nom**. C'est tout l'intérêt d'un Service.

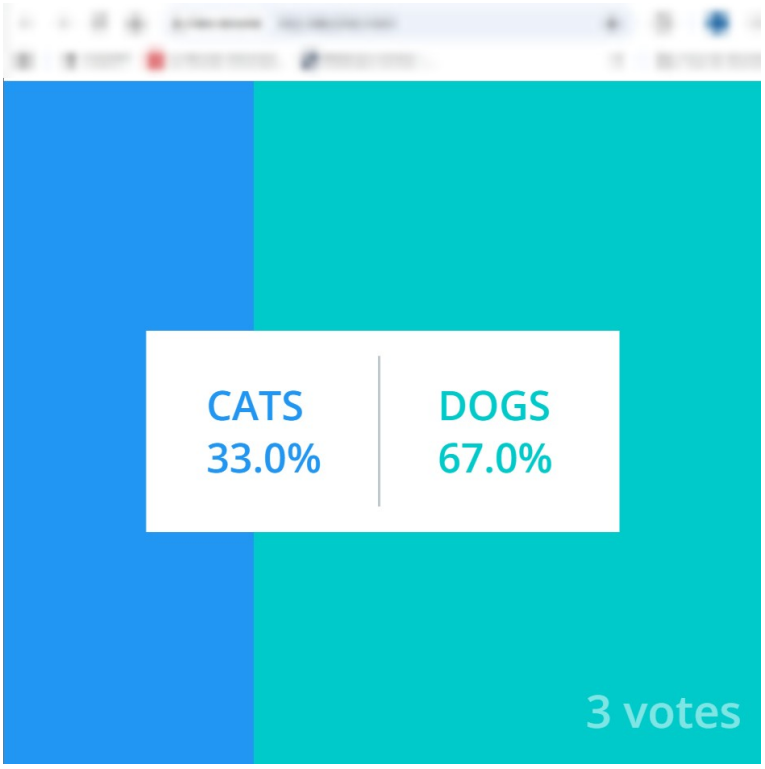


Figure 10 – Le vote remarche (le vote a été modifié et non ajouté)

On voit toujours 3 votes, mais la répartition a changé (33 % / 67 %). Les fenêtres privées ouvertes en même temps partagent leurs cookies : l'application a reconnu le même votant et a **modifié** son vote au lieu d'en ajouter un.

Partie 3 – L'Ingress : un point d'entrée avec de vrais noms

Les NodePorts, c'est pratique pour tester, mais ce n'est pas très propre : un port différent par application, et des numéros difficiles à retenir. Un **Ingress** permet d'avoir une seule porte d'entrée et de trier le trafic selon le nom du site demandé. Il faut pour cela un **contrôleur d'Ingress** : j'ai installé ingress-nginx.

Installation du contrôleur

VM 1

```
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.13.0/
deploy/static/provider/baremetal/deploy.yaml
kubectl -n ingress-nginx wait --for=condition=available deployment/ingress-nginx-
controller --timeout=180s
```

Le contrôleur reçoit des ports au hasard. Je les ai fixés à **30080** (HTTP) et **30443** (HTTPS) avec un petit fichier de modification :

ingress-nginx-nodeport-patch.yaml

```
spec:
  ports:
  - name: http
    port: 80
    targetPort: http
    nodePort: 30080
  - name: https
    port: 443
    targetPort: https
    nodePort: 30443
```

VM 1

```
kubectl -n ingress-nginx patch service ingress-nginx-controller --patch-file
ingress-nginx-nodeport-patch.yaml
kubectl -n ingress-nginx get svc ingress-nginx-controller
```

```
qassim@k8s-cp:~/tp3$ kubectl -n ingress-nginx get svc ingress-nginx-controll
er
NAME                       TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)
                  AGE
ingress-nginx-controller   NodePort   10.101.8.227   <none>        80:30080/
TCP,443:30443/TCP   116s
qassim@k8s-cp:~/tp3$
```

Figure 11 – Le contrôleur écoute bien sur 30080 et 30443

Création de l'Ingress

06-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: voting-app
  namespace: vote
spec:
  ingressClassName: nginx
  rules:
  - host: vote.local
```

```
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: vote
            port:
              number: 80
  - host: result.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: result
            port:
              number: 80
```

Ma première tentative a échoué :

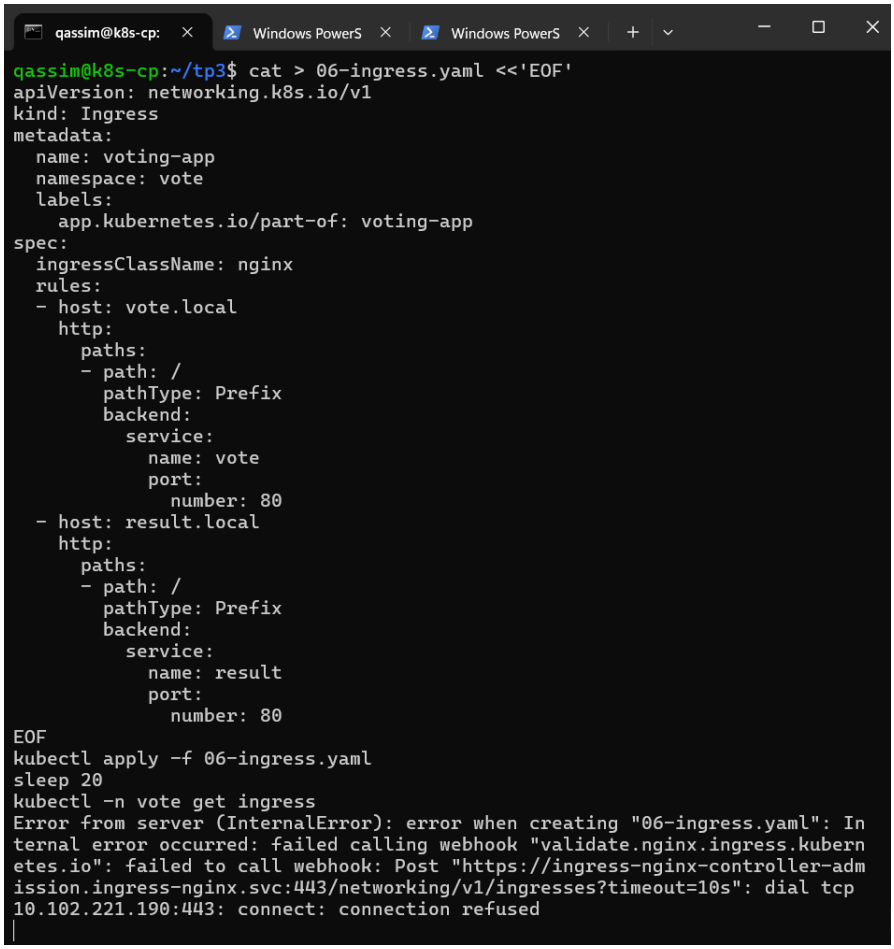


Figure 12 – Premier essai refusé par le webhook d'ingress-nginx

ingress-nginx installe un **webhook de validation** : avant d'accepter un Ingress, l'API demande au contrôleur de le vérifier. J'ai lancé la création trop tôt, alors que le contrôleur était encore en train de démarrer, d'où le `connection refused`. J'ai attendu qu'il soit prêt :

```
qassim@k8s-cp:~/tp3$ kubectl -n ingress-nginx get pods -o wide
kubectl -n ingress-nginx wait --for=condition=available deployment/ingress-n
ginx-controller --timeout=180s
kubectl -n ingress-nginx get endpointslices
NAME                                       READY   STATUS    RESTARTS   AGE
  IP              NODE     NOMINATED NODE   READINESS GATES
ingress-nginx-controller-7fb648f87-2cqqv   1/1     Running   0          64s
  10.244.228.80   k8s-w1   <none>           <none>
deployment.apps/ingress-nginx-controller condition met
NAME                                       ADDRESSTYPE   PORTS     ENDPOINTS
      AGE
ingress-nginx-controller-admission-xjzrl   IPv4          8443      10.244.228
.80   64s
ingress-nginx-controller-xq549             IPv4          443,80    10.244.228
.80   64s
qassim@k8s-cp:~/tp3$
```

Figure 13 – Le contrôleur est prêt et son webhook a un endpoint

```
qassim@k8s-cp:~/tp3$ kubectl -n ingress-nginx get pods -o wide
kubectl -n ingress-nginx wait --for=condition=available deployment/ingress-n
ginx-controller --timeout=180s
kubectl -n ingress-nginx get endpointslices
NAME                                       READY   STATUS    RESTARTS   AGE
  IP              NODE     NOMINATED NODE   READINESS GATES
ingress-nginx-controller-7fb648f87-2cqqv   1/1     Running   0          64s
  10.244.228.80   k8s-w1   <none>           <none>
deployment.apps/ingress-nginx-controller condition met
NAME                                       ADDRESSTYPE   PORTS     ENDPOINTS
      AGE
ingress-nginx-controller-admission-xjzrl   IPv4          8443      10.244.228
.80   64s
ingress-nginx-controller-xq549             IPv4          443,80    10.244.228
.80   64s
qassim@k8s-cp:~/tp3$ kubectl -n ingress-nginx get svc ingress-nginx-controll
er
NAME                       TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)
                  AGE
ingress-nginx-controller   NodePort   10.101.8.227   <none>        80:30080/
TCP,443:30443/TCP   116s
qassim@k8s-cp:~/tp3$ kubectl get ingressclass
kubectl apply -f 06-ingress.yaml
sleep 20
kubectl -n vote get ingress
NAME    CONTROLLER            PARAMETERS   AGE
nginx   k8s.io/ingress-nginx   <none>       2m52s
ingress.networking.k8s.io/voting-app created
NAME         CLASS   HOSTS                    ADDRESS   PORTS   AGE
voting-app   nginx   vote.local,result.local             80      20s
qassim@k8s-cp:~/tp3$
```

Figure 14 – L'IngressClass nginx existe et l'Ingress est créé

Cette fois, l'Ingress `voting-app` est créé avec la classe `nginx` et les deux noms `vote.local` et `result.local`.

Accès depuis mon PC

Mon PC ne connaît pas les noms `vote.local` et `result.local`. Je les ai ajoutés dans le fichier hosts de Windows (Bloc-notes lancé en administrateur) :

C:\Windows\System32\drivers\etc\hosts

```
10.2.100.219 vote.local result.local
```

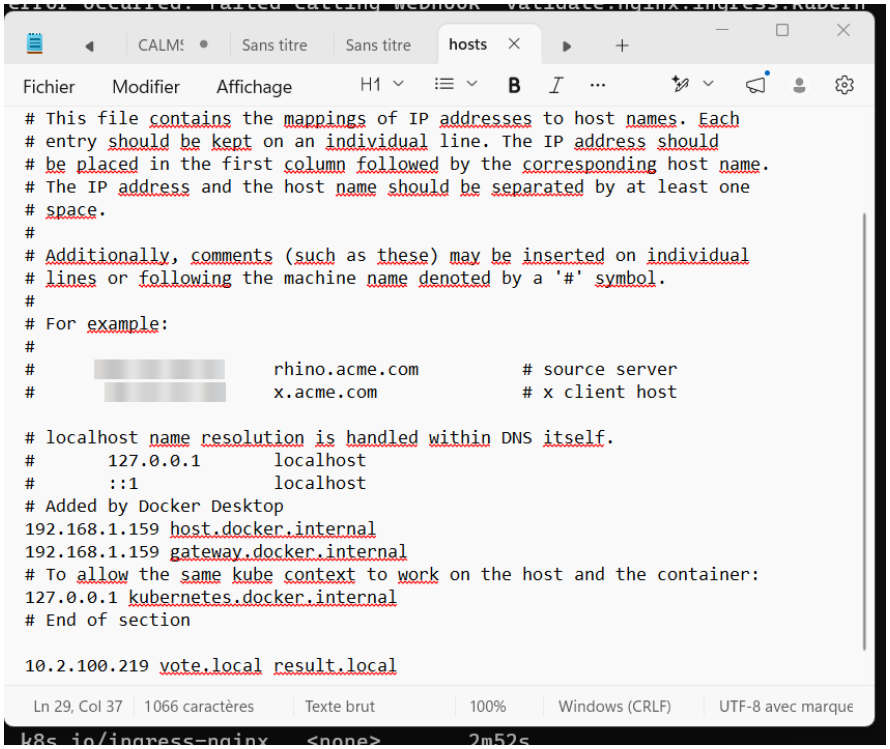


Figure 15 – Le fichier hosts de Windows avec la nouvelle ligne

Les deux sites ont ensuite répondu sur **http://vote.local:30080** et **http://result.local:30080** (on les retrouve dans les captures des parties suivantes). J'ai alors supprimé les NodePorts temporaires, devenus inutiles.

VM 1

```
kubectl delete -f 05b-nodeport-temporaire.yaml
kubectl -n vote get svc
curl -s http://10.2.100.219:30080/ | head -5
curl -s -H "Host: vote.local" http://10.2.100.219:30080/ | head -5
```

```
qassim@k8s-cp:~/tp3$ kubectl get ingressclass
kubectl apply -f 06-ingress.yaml
sleep 20
kubectl -n vote get ingress
NAME    CONTROLLER             PARAMETERS   AGE
nginx   k8s.io/ingress-nginx   <none>       2m52s
ingress.networking.k8s.io/voting-app created
NAME         CLASS   HOSTS                    ADDRESS   PORTS   AGE
voting-app   nginx   vote.local,result.local             80      20s
qassim@k8s-cp:~/tp3$ kubectl delete -f 05b-nodeport-temporaire.yaml
kubectl -n vote get svc
service "vote-np" deleted from vote namespace
service "result-np" deleted from vote namespace
NAME     TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
db       ClusterIP   10.103.193.148   <none>        5432/TCP   62m
redis    ClusterIP   10.102.155.132   <none>        6379/TCP   19m
result   ClusterIP   10.97.158.77     <none>        80/TCP     54m
vote     ClusterIP   10.104.126.191   <none>        80/TCP     54m
qassim@k8s-cp:~/tp3$ curl -s http://10.2.100.219:30080/ | head -5
curl -s -H "Host: vote.local" http://10.2.100.219:30080/ | head -5
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx</center>
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Cats vs Dogs!</title>
qassim@k8s-cp:~/tp3$
```

Figure 16 – NodePorts supprimés et test de la question 3

Question 3 – Avec et sans l'en-tête Host. Sans en-tête, `curl` demande en fait le site « 10.2.100.219 ». Aucune règle de l'Ingress ne correspond à ce nom : nginx répond **404 Not Found**. Avec `-H "Host: vote.local"`, la règle `vote.local` correspond, et on reçoit la page **Cats vs Dogs!**. L'Ingress trie donc le trafic selon le **nom du site demandé**, pas selon l'adresse IP. C'est ce qui permet de faire tourner plusieurs sites derrière une seule IP et un seul port, et c'est pour ça qu'il fallait modifier le fichier hosts.

Partie 4 – Mettre à jour, casser, revenir en arrière

Montée en charge et changement de configuration

J'ai passé vote à 3 copies (`kubectl scale … --replicas=3`), puis j'ai changé les options dans le fichier : **Thé / Café** à la place de Cats / Dogs, avec une nouvelle annotation `change-cause`.

VM 1

```
kubectl -n vote scale deployment/vote --replicas=3
kubectl -n vote rollout history deployment/vote     # une seule révision

sed -i -e 's/value: Cats/value: Thé/' -e 's/value: Dogs/value: Café/' \
       -e 's|version initiale : Cats / Dogs|options Thé / Café|' 03-vote.yaml
kubectl apply -f 03-vote.yaml
kubectl -n vote rollout status deployment/vote
```

Le `scale` ne crée **pas** de nouvelle révision : le nombre de copies ne fait pas partie de la « recette » des pods (le template). En revanche, changer une variable modifie la recette, donc Kubernetes crée une nouvelle version, avec un nouveau ReplicaSet.

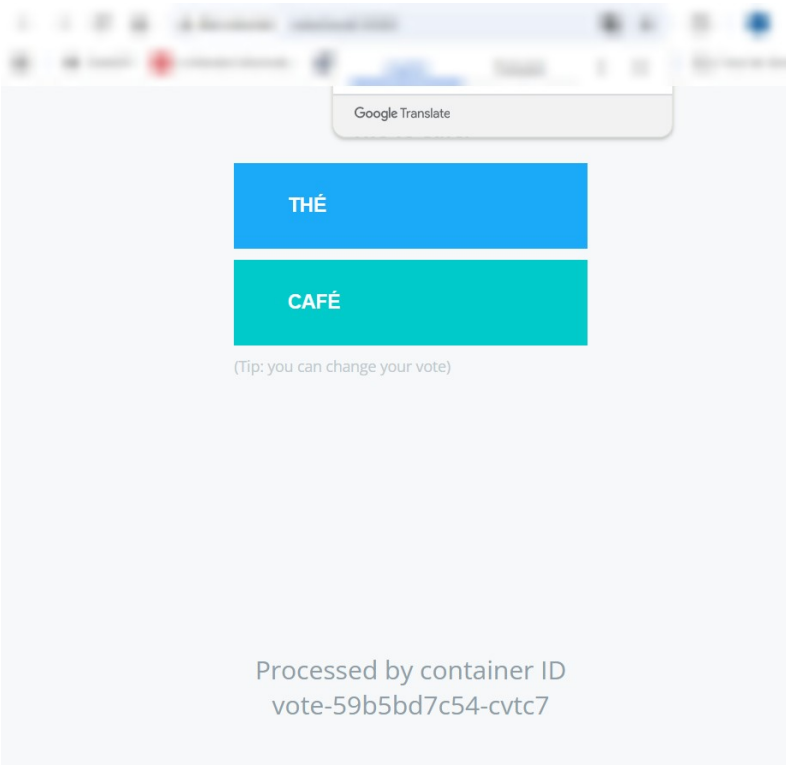


Figure 17 – La page de vote affiche maintenant Thé et Café

L'ordre des événements pendant la mise à jour, avec `maxSurge: 1` et `maxUnavailable: 0` : Kubernetes crée **un** nouveau pod (on a temporairement une copie de plus), attend qu'il soit prêt, puis seulement supprime **un** ancien pod. Il recommence ainsi pod par pod. On ne descend jamais en dessous du nombre de copies voulu, donc le site reste joignable du début à la fin.

En surveillant les pods avec `-w`, je me suis aperçu qu'il ne restait plus qu'**un seul pod vote**. Le fichier `03-vote.yaml` contenait encore `replicas: 1` : l'`apply` a remis la valeur du fichier, ou le scale n'avait pas été pris en compte. C'est justement le problème étudié à la fin de cette partie. J'ai simplement refait le scale à 3 avant la suite.

```
qassim@k8s-cp:~/tp3$ kubectl -n vote get pods -l app.kubernetes.io/nam
e=vote -w
NAME                    READY   STATUS    RESTARTS   AGE
vote-59b5bd7c54-cvtc7   1/1     Running   0          113s
```

Figure 18 – Un seul pod vote après l'apply : le fichier a imposé sa valeur

Panne volontaire

J'ai ensuite cassé l'application exprès, en demandant une image qui n'existe pas :

VM 1

```
kubectl -n vote set image deployment/vote
vote=dockersamples/examplevotingapp_vote:nexistepas
kubectl -n vote annotate deployment/vote kubernetes.io/change-cause="image
inexistante" --overwrite
kubectl -n vote rollout status deployment/vote --timeout=60s
kubectl -n vote get pods -l app.kubernetes.io/name=vote
```

```
qassim@k8s-cp:~/tp3$ kubectl -n vote set image deployment/vote vote=do
ckersamples/examplevotingapp_vote:nexistepas
kubectl -n vote annotate deployment/vote kubernetes.io/change-cause="i
mage inexistante" --overwrite
kubectl -n vote rollout status deployment/vote --timeout=60s
kubectl -n vote get pods -l app.kubernetes.io/name=vote
deployment.apps/vote image updated
deployment.apps/vote annotated
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
error: timed out waiting for the condition
NAME                    READY   STATUS         RESTARTS   AGE
vote-59b5bd7c54-bcfzm   1/1     Running        0          72s
vote-59b5bd7c54-cvtc7   1/1     Running        0          16m
vote-59b5bd7c54-nnc4v   1/1     Running        0          72s
vote-d4bdb5c86-bhjk2    0/1     ErrImagePull   0          60s
qassim@k8s-cp:~/tp3$
```

Figure 19 – Un nouveau pod en échec, les 3 anciens toujours en marche

Le déploiement reste bloqué (`timed out`). **Un seul** nouveau pod est créé, en `ErrImagePull` (l'image n'existe pas), et les **trois anciens pods continuent de tourner**. Le site répond toujours : `curl` renvoie le code **200**.

> **Pourquoi le site ne tombe pas ?** Grâce à `maxUnavailable: 0`, Kubernetes n'a pas le droit de supprimer un ancien pod tant que le nouveau n'est pas prêt. Le nouveau ne le sera jamais, donc aucun ancien pod n'est touché. Et grâce à `maxSurge: 1`, la casse se limite à un seul pod. C'est une vraie protection contre une mauvaise mise en production.

Retour arrière

VM 1

```
kubectl -n vote rollout undo deployment/vote
kubectl -n vote rollout status deployment/vote
kubectl -n vote rollout history deployment/vote
```

```
qassim@k8s-cp:~/tp3$ curl -s -o /dev/null -w "%{http_code}\n" -H "Host
: vote.local" http://10.2.100.219:30080/
200
qassim@k8s-cp:~/tp3$ kubectl -n vote rollout undo deployment/vote
kubectl -n vote rollout status deployment/vote
kubectl -n vote get pods -l app.kubernetes.io/name=vote
kubectl -n vote rollout history deployment/vote
deployment.apps/vote rolled back
Waiting for deployment spec update to be observed...
Waiting for deployment "vote" rollout to finish: 1 old replicas are pe
nding termination...
deployment "vote" successfully rolled out
NAME                    READY   STATUS        RESTARTS   AGE
vote-59b5bd7c54-bcfzm   1/1     Running       0          2m19s
vote-59b5bd7c54-cvtc7   1/1     Running       0          17m
vote-59b5bd7c54-nnc4v   1/1     Running       0          2m19s
vote-d4bdb5c86-bhjk2    0/1     Terminating   0          2m7s
deployment.apps/vote
REVISION  CHANGE-CAUSE
1         version initiale : Cats / Dogs
3         image inexistante
4         options Thé / Café

qassim@k8s-cp:~/tp3$
```

Figure 20 – Le site répond (200), retour arrière et historique des versions

Le pod en panne est supprimé, les trois bons pods n'ont jamais bougé. L'historique affiche les révisions **1**, **3** et **4**. La révision 2 semble avoir disparu : en réalité, `rollout undo` ne « recule » pas. Il crée une **nouvelle** version (la 4), qui est une copie de la 2. L'historique avance toujours, il ne revient jamais en arrière.

Le fichier est-il encore la vérité ?

VM 1

```
kubectl diff -f 03-vote.yaml
```

```
30:         image: dockersamples/examplevotingapp_vote
diff -u -N /tmp/LIVE-1205351797/apps.v1.Deployment.vote.vote /tmp/MERG
ED-2782683946/apps.v1.Deployment.vote.vote
--- /tmp/LIVE-1205351797/apps.v1.Deployment.vote.vote   2026-10-27 09:
33:53.978283409 +0000
+++ /tmp/MERGED-2782683946/apps.v1.Deployment.vote.vote 2026-10-27 09:
33:53.979283416 +0000
@@ -7,7 +7,7 @@
       {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annota
tions":{"kubernetes.io/change-cause":"options Thé / Café"},"labels":{"
app.kubernetes.io/name":"vote","app.kubernetes.io/part-of":"voting-app
"},"name":"vote","namespace":"vote"},"spec":{"replicas":1,"revisionHis
toryLimit":5,"selector":{"matchLabels":{"app.kubernetes.io/name":"vote
"}},"strategy":{"rollingUpdate":{"maxSurge":1,"maxUnavailable":0},"typ
e":"RollingUpdate"},"template":{"metadata":{"labels":{"app.kubernetes.
io/name":"vote","app.kubernetes.io/part-of":"voting-app"}},"spec":{"co
ntainers":[{"env":[{"name":"OPTION_A","value":"Thé"},{"name":"OPTION_B
","value":"Café"}],"image":"dockersamples/examplevotingapp_vote","name
":"vote","ports":[{"containerPort":80,"name":"http"}]}]}}}}
     kubernetes.io/change-cause: options Thé / Café
   creationTimestamp: "2026-10-27T08:18:53Z"
-  generation: 7
+  generation: 8
   labels:
     app.kubernetes.io/name: vote
     app.kubernetes.io/part-of: voting-app
@@ -17,7 +17,7 @@
   uid: e2fa12c7-1494-4c99-8eff-3ff5b3532a7f
 spec:
   progressDeadlineSeconds: 600
-  replicas: 3
+  replicas: 1
   revisionHistoryLimit: 5
   selector:
     matchLabels:
qassim@k8s-cp:~/tp3$
```

Figure 21 – kubectl diff : le cluster a 3 copies, le fichier en demande 1

> **Question 4.6 – Le cluster et le fichier sont-ils identiques ?** Non. `kubectl diff` montre que le cluster tourne avec **3 copies** alors que le fichier n'en demande qu'**une**. Les commandes `scale` et `set image` ont changé le cluster sans toucher au fichier. Si je relance `kubectl apply`, vote retombera à une seule copie. Pour que le fichier redevienne la **source de vérité**, il faut y écrire l'état voulu, puis **ne plus jamais modifier le cluster à la main**. Au jour 3, le GitOps (Argo CD ou Flux) le fera automatiquement : un outil compare en permanence le cluster avec Git et corrige les écarts.

J'ai donc mis `replicas: 3` dans le fichier. `kubectl diff` ne montre alors plus aucune différence :

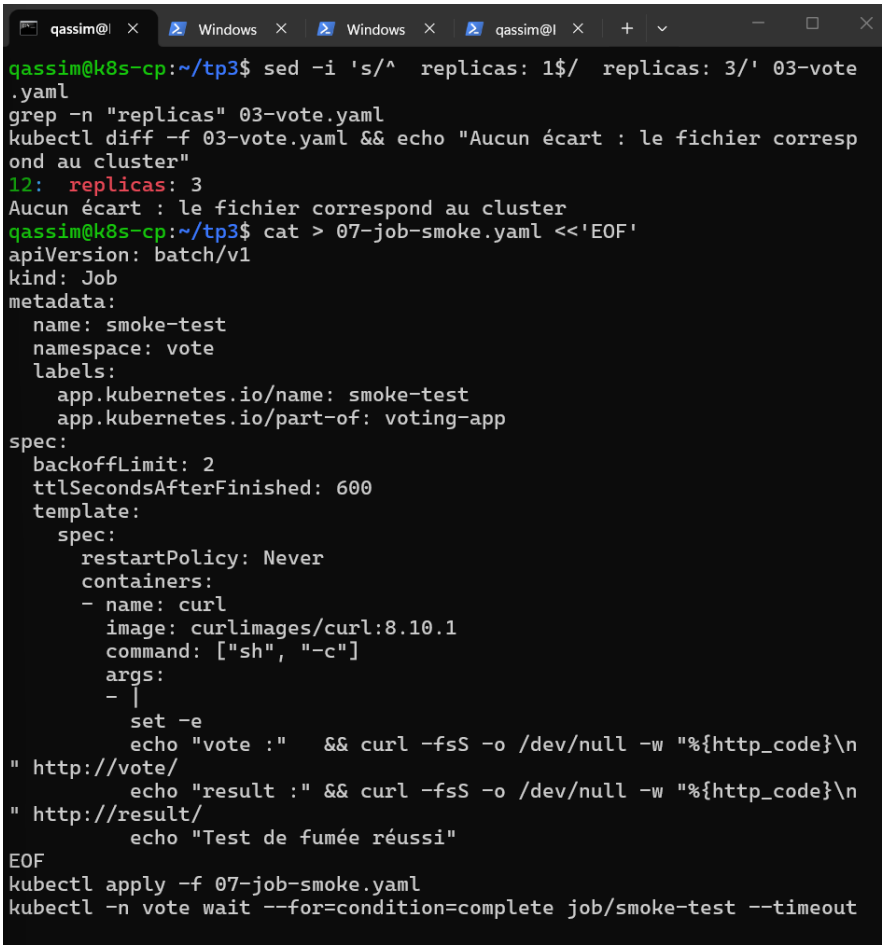


Figure 22 – Après correction du fichier : aucun écart avec le cluster

Partie 5 – Les tâches ponctuelles : Job et CronJob

Job : un test de bon fonctionnement

Un **Job** lance un conteneur qui fait une tâche et s'arrête, contrairement à un Deployment qui tourne en continu. J'en ai écrit un qui vérifie que les pages vote et result répondent bien, depuis l'intérieur du cluster.

07-job-smoke.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: smoke-test
  namespace: vote
spec:
  backoffLimit: 2
  ttlSecondsAfterFinished: 600
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: curl
        image: curlimages/curl:8.10.1
        command: ["sh", "-c"]
        args:
        - |
          set -e
          echo "vote :"   && curl -fsS -o /dev/null -w "%{http_code}\n"
http://vote/
          echo "result :" && curl -fsS -o /dev/null -w "%{http_code}\n"
http://result/
          echo "Test de fumée réussi"
```

`curl -f` s'arrête en erreur si la page ne répond pas correctement, et `set -e` fait échouer tout le script dans ce cas. Le Job réessaie alors jusqu'à 2 fois (`backoffLimit`). `ttlSecondsAfterFinished: 600` le supprime automatiquement 10 minutes après la fin.

CronJob : une tâche planifiée

08-cronjob-menage.yaml

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: menage
  namespace: vote
spec:
  schedule: "*/1 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
```

```
        spec:
          restartPolicy: OnFailure
          containers:
          - name: menage
            image: busybox:1.36
            command: ["sh", "-c", "echo \"Ménage lancé le $(date)\"; sleep 5;
echo \"Ménage terminé\""]
```

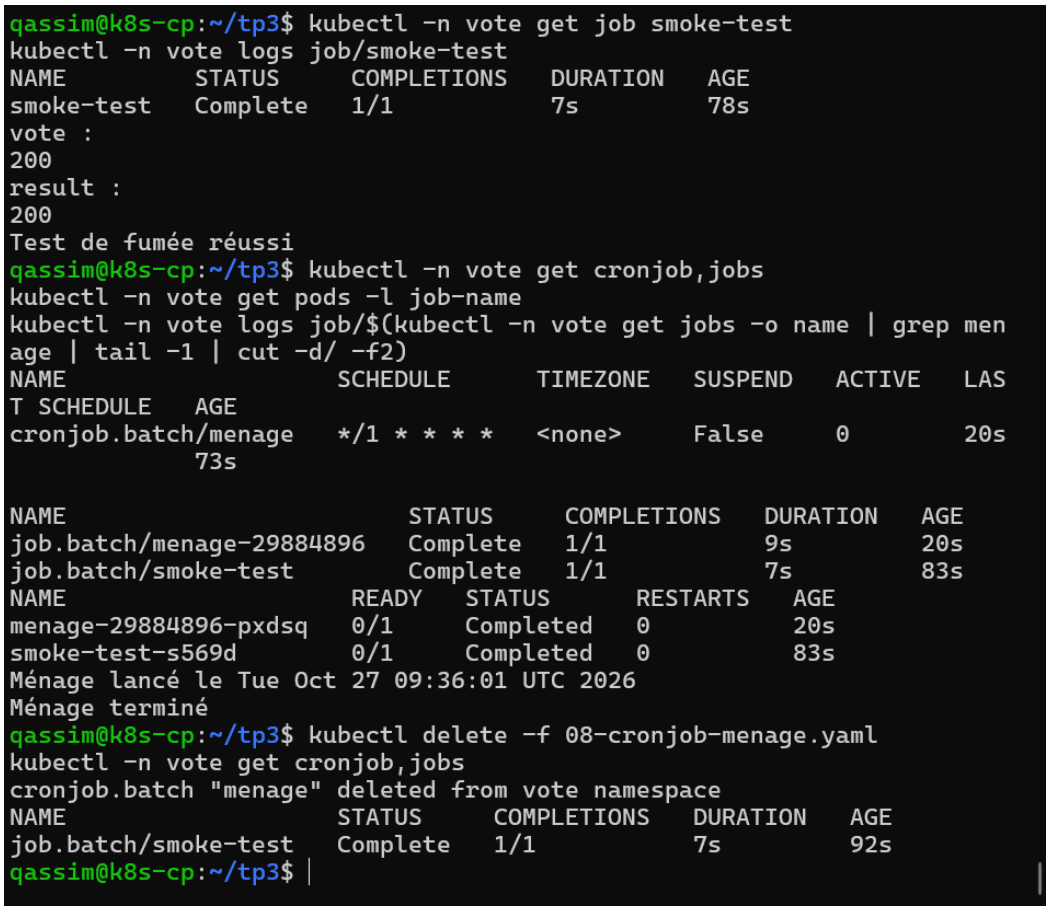


Figure 23 – Job réussi (vote 200, result 200) et premier passage du CronJob

Le Job est **Complete** en 7 secondes, avec `vote : 200` et `result : 200`. J'avais supprimé le CronJob trop tôt la première fois (il n'avait tourné qu'une fois). Je l'ai relancé et laissé tourner 4 minutes :

```
qassim@k8s-cp:~/tp3$ kubectl apply -f 08-cronjob-menage.yaml
echo "Attente de 4 minutes..." && sleep 240
kubectl -n vote get cronjob,jobs
kubectl -n vote get pods -l job-name
cronjob.batch/menage created
Attente de 4 minutes...
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAS
T SCHEDULE   AGE
cronjob.batch/menage   */1 * * * *   <none>     False     1        7s
             4m

NAME                         STATUS     COMPLETIONS   DURATION   AGE
job.batch/menage-29884899    Complete   1/1           8s         2m7s
job.batch/menage-29884900    Complete   1/1           9s         67s
job.batch/menage-29884901    Running    0/1           7s         7s
job.batch/smoke-test         Complete   1/1           7s         6m10s
NAME                    READY   STATUS      RESTARTS   AGE
menage-29884899-dmsk5   0/1     Completed   0          2m7s
menage-29884900-m6857   0/1     Completed   0          67s
menage-29884901-kd6jd   0/1     Completed   0          7s
smoke-test-s569d        0/1     Completed   0          6m10s
qassim@k8s-cp:~/tp3$
```

Figure 24 – Après 4 minutes : seuls les 2 derniers Jobs réussis sont gardés

Le CronJob crée **un Job par minute** (les numéros se suivent : 899, 900, 901). Pourtant, seuls les **deux derniers Jobs réussis** sont gardés, plus celui en cours : le plus ancien a été supprimé automatiquement grâce à `successfulJobsHistoryLimit: 2`. Sans cette limite, des centaines de Jobs terminés s'accumuleraient. `concurrencyPolicy: Forbid` empêche aussi de lancer une exécution tant que la précédente n'est pas finie. J'ai ensuite supprimé le CronJob pour ne pas encombrer le namespace.

> **Question 4 – Pourquoi un Job ne peut-il pas avoir `restartPolicy: Always` ?** Un Job attend que son conteneur **se termine** pour se déclarer réussi. Avec `Always`, Kubernetes relancerait le conteneur à chaque fois qu'il s'arrête, même après un succès : le Job ne finirait jamais. L'API refuse d'ailleurs cette valeur : un Job n'accepte que `Never` (un nouveau pod à chaque essai) ou `OnFailure` (on relance le même pod seulement en cas d'erreur).

Partie 6 – Le constat : les votes ne survivent pas

Dernière expérience, et la plus parlante. Avant, la page des résultats affichait **3 votes** :

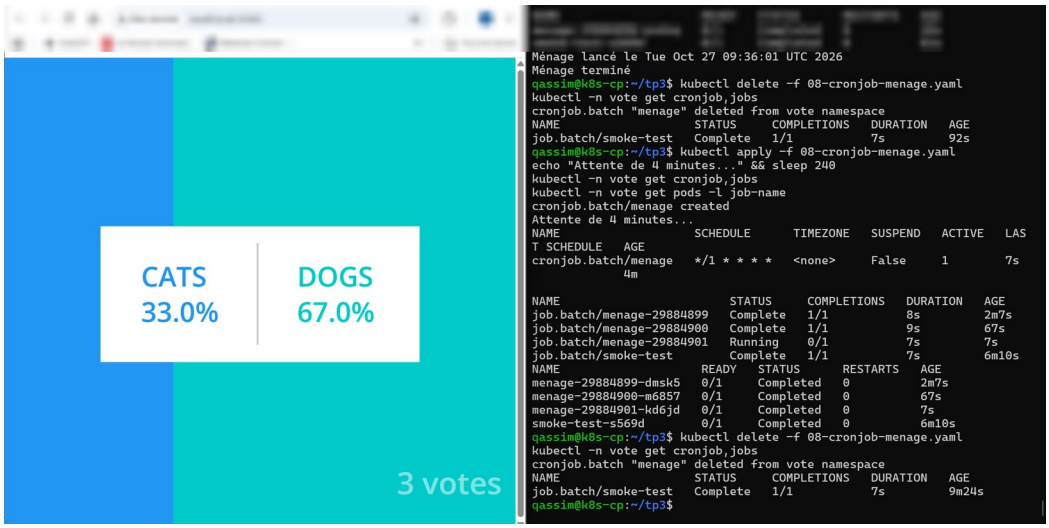


Figure 25 – Avant : 3 votes enregistrés

J'ai supprimé le pod de la base de données, et laissé Kubernetes le recréer :

VM 1

```
kubectl -n vote delete pod -l app.kubernetes.io/name=db
kubectl -n vote get pods -l app.kubernetes.io/name=db -w
```

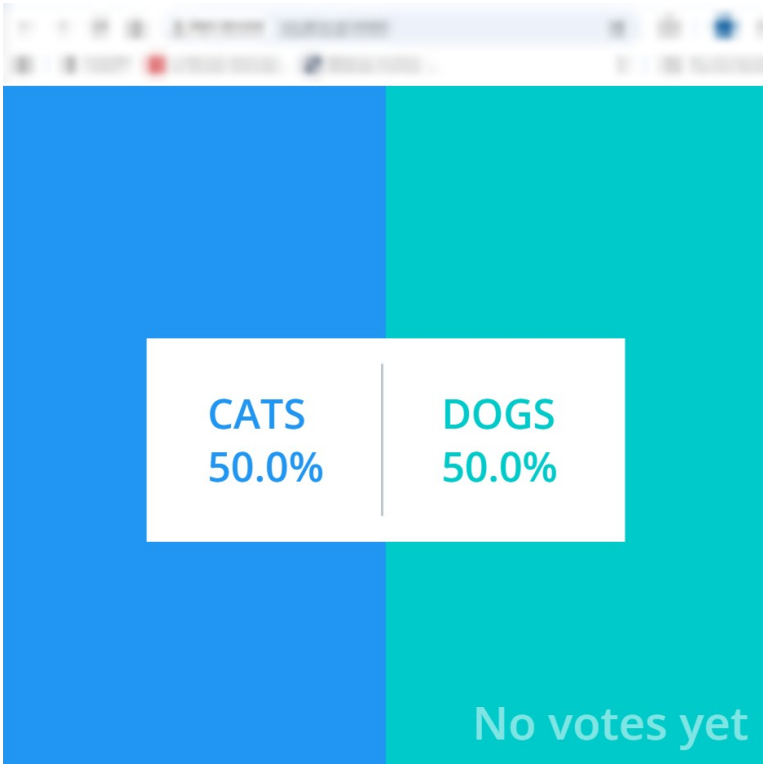


Figure 26 – Après : plus aucun vote

Le résultat est sans appel : **« No votes yet »**. Les trois votes ont disparu.

```
qassim@k8s-cp:~/tp3$ kubectl -n vote get pods
kubectl -n vote logs deploy/worker --tail=5
NAME                      READY   STATUS    RESTARTS      AGE
db-596fbb45b6-rd7n8       1/1     Running   0             71s
redis-54d64cc577-4qxp4    1/1     Running   0             96m
result-556df44dc7-x9cwp   1/1     Running   1 (71s ago)   87m
vote-59b5bd7c54-bcfzm     1/1     Running   0             15m
vote-59b5bd7c54-cvtc7     1/1     Running   0             30m
vote-59b5bd7c54-nnc4v     1/1     Running   0             15m
worker-66d54b4447-9m56k   1/1     Running   1 (71s ago)   87m
Waiting for db
Waiting for db
Connected to db
Found redis at 10.102.155.132
Connecting to redis
qassim@k8s-cp:~/tp3$
```

Figure 27 – Le nouveau pod db, et le worker qui s'est reconnecté

Kubernetes a bien recréé un pod db (nouveau nom, 71 secondes d'âge). Le **worker** et **result** ont redémarré une fois au même moment : ils avaient perdu la connexion à la base. Les logs du worker montrent qu'il attend la base (`Waiting for db`), s'y reconnecte, puis retrouve redis. L'application refonctionne : un nouveau vote est bien compté.

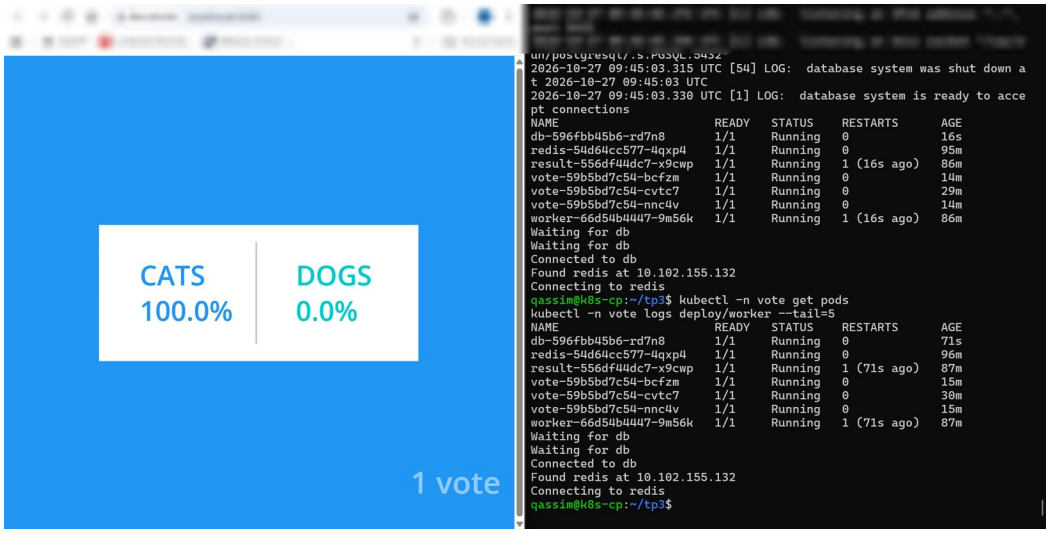


Figure 28 – L'application repart de zéro : 1 nouveau vote

On remarque que result affiche « Cats » alors que j'ai voté sur la page Thé / Café. Les libellés de result sont écrits en dur dans son code : il ne lit pas `OPTION_A` et `OPTION_B`. La base enregistre seulement « a » ou « b ».

> **Question 5 – Que sont devenus les votes ?** Ils sont **perdus**. La base rangeait ses données dans un volume **emptyDir**, qui vit et meurt avec le pod. En supprimant le pod, Kubernetes a aussi effacé ce dossier. Le Deployment a bien recréé un pod, mais avec un dossier vide : PostgreSQL est reparti d'une base neuve. Un Deployment garantit qu'un pod tourne, **pas que les données soient conservées**.
>
> **Les deux objets du TP4 qui corrigeront cela :** un **PersistentVolumeClaim** (une demande de disque durable, indépendant du pod) et un **StatefulSet** (à la place du Deployment), qui donne au pod une identité fixe, `db-0`, toujours reliée au même disque.

Partie 7 – Pour aller plus loin

Canary à la main

Un **canary** consiste à envoyer une petite partie des utilisateurs vers une nouvelle version, pour la tester sur un échantillon avant de la généraliser. J'ai créé un second Deployment, `vote-canary`, avec des options différentes (Chocolat / Café).

bonus-vote-canary.yaml (extrait)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vote-canary
  namespace: vote
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: vote
      track: canary
  template:
    metadata:
      labels:
        app.kubernetes.io/name: vote
        app.kubernetes.io/part-of: voting-app
        track: canary
    spec:
      containers:
      - name: vote
        image: dockersamples/examplevotingapp_vote
        env:
        - name: OPTION_A
          value: Chocolat
        - name: OPTION_B
          value: Café
```

L'astuce est dans les étiquettes : les pods du canary portent `app.kubernetes.io/name: vote`, donc le **Service vote les prend aussi**. L'étiquette en plus, `track: canary`, sert uniquement au Deployment canary pour ne gérer que ses propres pods. J'ai mesuré la répartition avec 40 requêtes :

VM 1

```
for i in $(seq 1 40); do
  curl -s -H "Host: vote.local" http://10.2.100.219:30080/ | grep -o "Chocolat\|
Thé" | head -1
done | sort | uniq -c
```

```
qassim@k8s-cp:~/tp3$ for i in $(seq 1 40); do
  curl -s -H "Host: vote.local" http://10.2.100.219:30080/ | grep -o "
Chocolat\|Thé" | head -1
done | sort | uniq -c
     10 Chocolat
     30 Thé
```

Figure 29 – 3 pods stables et 1 canary : 1 requête sur 4 va vers le canary

```
qassim@k8s-cp:~/tp3$ kubectl -n vote scale deployment/vote-canary --re
plicas=3
kubectl -n vote rollout status deployment/vote-canary
for i in $(seq 1 40); do
  curl -s -H "Host: vote.local" http://10.2.100.219:30080/ | grep -o "
Chocolat\|Thé" | head -1
done | sort | uniq -c
deployment.apps/vote-canary scaled
Waiting for deployment "vote-canary" rollout to finish: 1 out of 3 new
 replicas have been updated...
Waiting for deployment "vote-canary" rollout to finish: 1 of 3 updated
 replicas are available...
Waiting for deployment "vote-canary" rollout to finish: 2 of 3 updated
 replicas are available...
deployment "vote-canary" successfully rolled out
     16 Chocolat
     24 Thé
```

Figure 30 – 3 pods stables et 3 canary : la part du canary augmente

Pods stables / canary	Part attendue	Mesure sur 40 requêtes
3 / 1	25 %	10 Chocolat sur 40 = **25 %**
3 / 3	50 %	16 Chocolat sur 40 = **40 %**

La part du canary dépend simplement du **nombre de pods** de chaque côté. L'écart du second essai (40 % au lieu de 50 %) s'explique par le petit nombre de requêtes et par une mesure lancée juste après l'arrivée des nouveaux pods. Cette méthode a une limite : pour envoyer seulement 1 % des utilisateurs, il faudrait 99 pods stables pour 1 canary. Pour doser finement, on utilise des outils dédiés (annotation `canary-weight` d'ingress-nginx, Argo Rollouts ou Gateway API). J'ai ensuite supprimé le canary.

MetalLB : volontairement non réalisé

MetalLB permet de donner une vraie adresse IP à un Service, sans NodePort. Pour cela, il « réserve » des adresses sur le réseau local en répondant aux requêtes ARP. Avant de l'installer, j'ai vérifié si les adresses choisies étaient libres :

```
qassim@k8s-cp:~/tp3$ for ip in 10.2.100.240 10.2.100.241 10.2.100.242;
 do
  ping -c 1 -W 1 $ip >/dev/null && echo "$ip OCCUPÉE" || echo "$ip lib
re (pas de réponse)"
done
10.2.100.240 OCCUPÉE
10.2.100.241 libre (pas de réponse)
10.2.100.242 OCCUPÉE
qassim@k8s-cp:~/tp3$
```

Figure 31 – Deux des trois adresses testées sont déjà utilisées

Deux adresses sur trois étaient **déjà utilisées**. Surtout, mes deux VM ont reçu leur IP en 10.2.100.x par DHCP : cette plage fait donc partie des adresses distribuées automatiquement par le réseau de l'école. Même une adresse libre aujourd'hui pourrait être donnée demain à un autre poste, ce qui créerait un **conflit d'adresses** gênant pour d'autres personnes. L'énoncé précise d'ailleurs que la plage doit être libre **et hors DHCP**. J'ai donc choisi de ne pas faire ce bonus sans une plage validée par le formateur.

Avec une plage libre, la procédure aurait été : installer le manifest `metallb-native.yaml`, déclarer un `IPAddressPool` et une `L2Advertisement`, puis créer un Service `vote-lb` de type `LoadBalancer`. Il aurait reçu une IP externe du pool, joignable directement depuis le PC, sans port ni fichier hosts.

Livrables et état final

Historique du déploiement vote

VM 1

```
kubectl -n vote rollout history deployment/vote
kubectl -n vote get deploy,svc,ingress
```

```
qassim@k8s-cp:~$ cd ~/tp3
kubectl delete -f bonus-vote-canary.yaml --ignore-not-found
kubectl -n vote rollout history deployment/vote
kubectl -n vote get deploy,svc,ingress
deployment.apps/vote
REVISION  CHANGE-CAUSE
1         version initiale : Cats / Dogs
3         image inexistante
4         options Thé / Café

NAME                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/db       1/1     1            1           126m
deployment.apps/redis    1/1     1            1           126m
deployment.apps/result   1/1     1            1           117m
deployment.apps/vote     3/3     3            3           117m
deployment.apps/worker   1/1     1            1           117m

NAME             TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
 AGE
service/db       ClusterIP   10.103.193.148   <none>        5432/TCP
 126m
service/redis    ClusterIP   10.102.155.132   <none>        6379/TCP
 82m
service/result   ClusterIP   10.97.158.77     <none>        80/TCP
 117m
service/vote     ClusterIP   10.104.126.191   <none>        80/TCP
 117m

NAME                                    CLASS   HOSTS
   ADDRESS        PORTS   AGE
ingress.networking.k8s.io/voting-app   nginx   vote.local,result.local
   10.2.100.219   80      76m
qassim@k8s-cp:~/tp3$
```

Figure 32 – Historique des versions et état final de l'application

Révision	CHANGE-CAUSE
1	version initiale : Cats / Dogs
3	image inexistante
4	options Thé / Café

L'état final est propre : 5 Deployments disponibles (vote en 3/3), 4 Services ClusterIP et l'Ingress `voting-app` avec ses deux noms. L'Ingress affiche maintenant l'adresse 10.2.100.219, celle du nœud où tourne le contrôleur.

Fichiers du TP

J'ai copié tous mes fichiers sur mon PC, pour les versionner dans Git au jour 3 :

PowerShell (PC)

```
scp -r qassim@10.2.100.225:~/tp3 "$env:USERPROFILE\Documents\TP3"
```

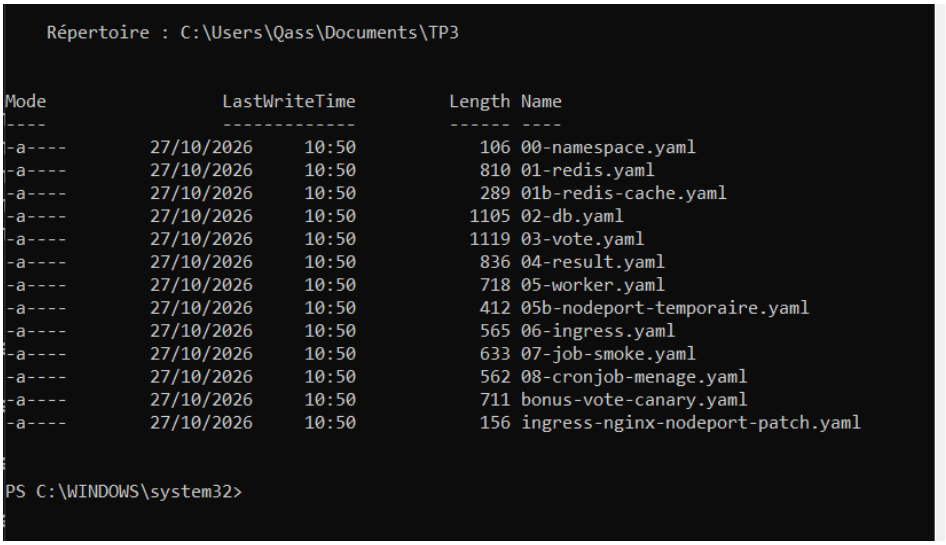


Figure 33 – Les fichiers YAML du TP3 copiés sur le PC

Fichier	Contenu
`00-namespace.yaml`	Namespace vote
`01-redis.yaml`	Deployment et Service redis
`02-db.yaml`	Deployment (Recreate, emptyDir) et Service db
`03-vote.yaml`	Deployment (RollingUpdate, 3 copies) et Service vote
`04-result.yaml`	Deployment et Service result
`05-worker.yaml`	Deployment worker, sans Service (commentaire explicatif)
`06-ingress.yaml`	Ingress vote.local / result.local
`07-job-smoke.yaml`	Job de test
`08-cronjob-menage.yaml`	CronJob chaque minute
Fichiers annexes	`01b-redis-cache.yaml`, `05b-nodeport-temporaire.yaml`, `ingress-nginx-nodeport-patch.yaml`, `bonus-vote-canary.yaml`

Difficultés rencontrées

Problème	Cause	Solution
Archive TP3.zip illisible	Fichier abîmé par la plateforme	Écriture de tous les fichiers YAML moi-même
Coupures SSH	Connexion inactive	Option `ServerAliveInterval=30` et reconnexion
Ingress refusé (webhook)	Contrôleur pas encore démarré	Attendre qu'il soit prêt, puis relancer
Fichier hosts invisible	Filtre « *.txt » du Bloc-notes	Choisir « Tous les fichiers »
`ipconfig /flushdns` refusé	Droits administrateur requis	Pas nécessaire : Windows relit le fichier hosts
vote retombé à 1 copie	Le fichier imposait `replicas: 1`	Nouveau scale, puis fichier corrigé à 3
CronJob supprimé trop tôt	Observation après 1 minute seulement	Relance et observation après 4 minutes
Pas de plage IP libre pour MetalLB	Réseau d'école en DHCP	Bonus non réalisé, choix expliqué

Conclusion

Ce TP m'a permis de passer de « faire tourner des pods » à « faire tourner une application ». J'ai compris que chaque composant a besoin de deux choses : un **Deployment** pour exister et se relancer, et un **Service** pour être trouvé par son nom. J'ai vu concrètement que ce nom est un vrai contrat avec le code : le changer suffit à casser l'application.

L'Ingress m'a montré comment publier plusieurs sites derrière une seule adresse, en triant selon le nom demandé. La partie rollout m'a le plus marqué : grâce à `maxUnavailable: 0`, même une mise à jour complètement ratée n'a jamais coupé le site, et un simple `rollout undo` a tout réparé. J'ai aussi compris le danger de modifier le cluster à la main : le fichier n'est plus la vérité, et le prochain `apply` peut défaire ce qu'on a fait. C'est exactement ce que le GitOps viendra régler au jour 3.

Enfin, la disparition des votes après le redémarrage de la base m'a montré la limite des Deployments pour les données. Un pod est jetable, ses données ne doivent pas l'être : ce sera l'objet du TP4, avec les volumes persistants et les StatefulSets.